

Sistema de agendamento para oficinas mecânicas

Isael Soares dos Santos¹, Natan Patzlaff², Filipe G. Hollmann³, Luani Pereira Chaves⁴

¹Instituto Federal Catarinense – Campus Concórdia
Concórdia – SC – Brasil

Abstract. *This paper presents the development of an online scheduling system for mechanical workshops, designed to replace manual appointment management carried out through messaging applications. The proposed system allows clients to view available time slots and schedule services autonomously, while providing workshops with an administrative panel for agenda management. The solution was validated through a survey with clients and automotive professionals, confirming the relevance of the problem and the demand for automation. The system is modeled using UML class diagrams and follows SOLID principles to ensure maintainability and extensibility of the codebase.*

Resumo. *Este artigo apresenta o desenvolvimento de um sistema de agendamento online para oficinas mecânicas, com o objetivo de substituir o gerenciamento manual de atendimentos realizado por meio de aplicativos de mensagens. O sistema proposto permite que clientes visualizem horários disponíveis e realizem agendamentos de forma autônoma, enquanto oferece às oficinas um painel administrativo para o gerenciamento da agenda. A solução foi validada por meio de um formulário aplicado a clientes e profissionais do setor automotivo, confirmando a relevância do problema e a demanda pela automatização. O sistema é modelado com diagrama de classes UML e segue os princípios SOLID para garantir a manutenibilidade e extensibilidade do código.*

1. Definição do problema

1.1. Contexto do Problema

Atualmente, oficinas mecânicas realizam atendimentos principalmente por aplicativos de mensagem, como o WhatsApp. Isso faz com que clientes precisem esperar respostas apenas para verificar disponibilidade de horários, tornando o processo lento e pouco eficiente [SEBRAE 2023].

1.2. Público-Alvo

Oficinas mecânicas e centros automotivos, além de clientes que buscam praticidade no agendamento de serviços.

1.3. Justificativa

A falta de um sistema automatizado gera perda de tempo tanto para a empresa quanto para o cliente. Muitas mensagens recebidas são apenas para consulta de horários, sobrecarregando o atendimento.

O sistema permitirá que o cliente:

- Escolha o serviço;
- Visualize horários disponíveis;
- Faça o agendamento online.

1.4. Motivação para o Desenvolvimento

O projeto busca modernizar o processo de agendamento, reduzir o volume de atendimentos manuais e melhorar a organização da oficina. Além disso, oferece mais rapidez e comodidade para os clientes, atendendo uma necessidade real do setor automotivo.

2. Validação - Perguntas

Para validar o nosso problema e a relevância da proposta de solução, foi aplicado um formulário online direcionado a clientes de oficinas mecânicas e profissionais do setor automotivo. Ao todo, foram obtidas 12 respostas, sendo 11 clientes e 1 proprietário.

De acordo com os resultados coletados é identificado que o problema identificado possui relevância. Entre os clientes que responderam a pesquisa, 54,5% (6 pessoas) afirmaram já ter dificuldades para obter informações sobre horários disponíveis em oficinas mecânicas. Além disso, apenas 45,5% (5 pessoas) consideram o processo atual de agendamento eficiente, enquanto 45,5% classificaram como parcialmente eficiente e os outros 9% como ineficiente.

Outro dado relevante da pesquisa, foi que quando questionados sobre a utilização de um sistema online para agendamento de serviços automotivos, 72,7% dos participantes (8 pessoas) responderam que utilizariam a plataforma, no entanto apenas 18,2% (2 pessoas) responderam que não utilizariam.

A funcionalidade considerada mais relevante pelos usuários foi a possibilidade de visualizar horários disponíveis sem depender de atendimento manual. Todos os respondentes consideraram esse recurso útil, sendo que 63,6% (7 pessoas) o classificaram como “muito útil” e 36,4% (4 pessoas) como “útil”.

Em relação às oficinas, embora tenha sido obtida apenas uma resposta, os resultados reforçam a existência do problema. O profissional entrevistado informou que:

- O atendimento manual via WhatsApp atrapalha a organização da oficina;
- Já houve perda de clientes devido à demora no atendimento;
- Existe dificuldade na organização manual de horários e serviços;
- Um sistema online poderia contribuir para o crescimento e organização do negócio.

Com base nos dados coletados, conclui-se que a proposta de um sistema de agendamento online para oficinas mecânicas atende a uma necessidade real tanto dos clientes quanto das empresas do setor. Os resultados indicam interesse na automatização do processo de agendamento, especialmente pela praticidade de consultar horários disponíveis e reduzir a dependência do atendimento manual, validando assim o problema identificado e justificando o desenvolvimento da solução proposta.

3. Escopo do Sistema

3.1. Funcionalidades Principais

O sistema tem como objetivo automatizar o processo de agendamento de serviços automotivos, proporcionando mais praticidade para clientes e melhor organização para as oficinas [Sommerville 2019].

As principais funcionalidades do sistema são:

- Cadastro e autenticação de usuários, permitindo o acesso tanto de clientes quanto de oficinas mecânicas;
- Cadastro de veículos por meio da placa, com preenchimento automático das informações do veículo quando disponíveis;
- Visualização dos serviços oferecidos pela oficina;
- Consulta de horários disponíveis para atendimento;
- Realização de agendamentos de forma online;
- Cancelamento de agendamentos pelo cliente;
- Área administrativa para gerenciamento dos agendamentos;
- Cadastro, edição e cancelamento de agendamentos pela oficina;
- Visualização da agenda de atendimentos;
- Armazenamento e consulta do histórico de agendamentos realizados.

3.2. Requisitos Funcionais

- **RF01** – O sistema deve permitir o cadastro e autenticação de clientes e oficinas.
- **RF02** – O cliente deve poder cadastrar veículos utilizando a placa.
- **RF03** – O sistema deve exibir os serviços disponíveis para agendamento.
- **RF04** – O cliente deve visualizar os horários disponíveis antes de realizar um agendamento.
- **RF05** – O sistema deve permitir a realização de agendamentos online.
- **RF06** – O cliente deve poder consultar seus agendamentos.
- **RF07** – O cliente deve poder cancelar agendamentos futuros.
- **RF08** – A oficina deve poder visualizar todos os agendamentos.
- **RF09** – A oficina deve poder criar, editar e cancelar agendamentos.
- **RF10** – O sistema deve armazenar o histórico dos agendamentos realizados.

3.3. Regras de Negócio

- **RN01** – Um mesmo horário não poderá ser reservado por mais de um cliente.
- **RN02** – Apenas horários disponíveis poderão ser agendados.
- **RN03** – Todo agendamento deve estar associado a um veículo cadastrado.
- **RN04** – O cancelamento somente poderá ocorrer antes da data e horário do atendimento.
- **RN05** – Apenas oficinas cadastradas poderão gerenciar a agenda de atendimentos.
- **RN06** – Serviços que exijam análise prévia ou orçamento personalizado não estarão disponíveis para agendamento automático.
- **RN07** – O sistema deverá registrar a data e horário de criação dos agendamentos.

3.4. Limitações do Sistema

- O sistema será destinado inicialmente a oficinas mecânicas de pequeno e médio porte.
- O sistema não realizará pagamentos online.
- Serviços que dependam de avaliação técnica prévia não poderão ser agendados automaticamente.
- O sistema não realizará diagnósticos automotivos.
- O sistema não substitui a execução dos serviços da oficina, apenas o gerenciamento dos agendamentos.
- Integrações com WhatsApp, SMS e e-mail não fazem parte da versão inicial do projeto.

4. Modelagem UML

A modelagem do sistema foi realizada por meio de um diagrama de classes UML [Booch et al. 2005], representando as principais entidades do domínio e seus relacionamentos, conforme ilustrado na Figura 1.

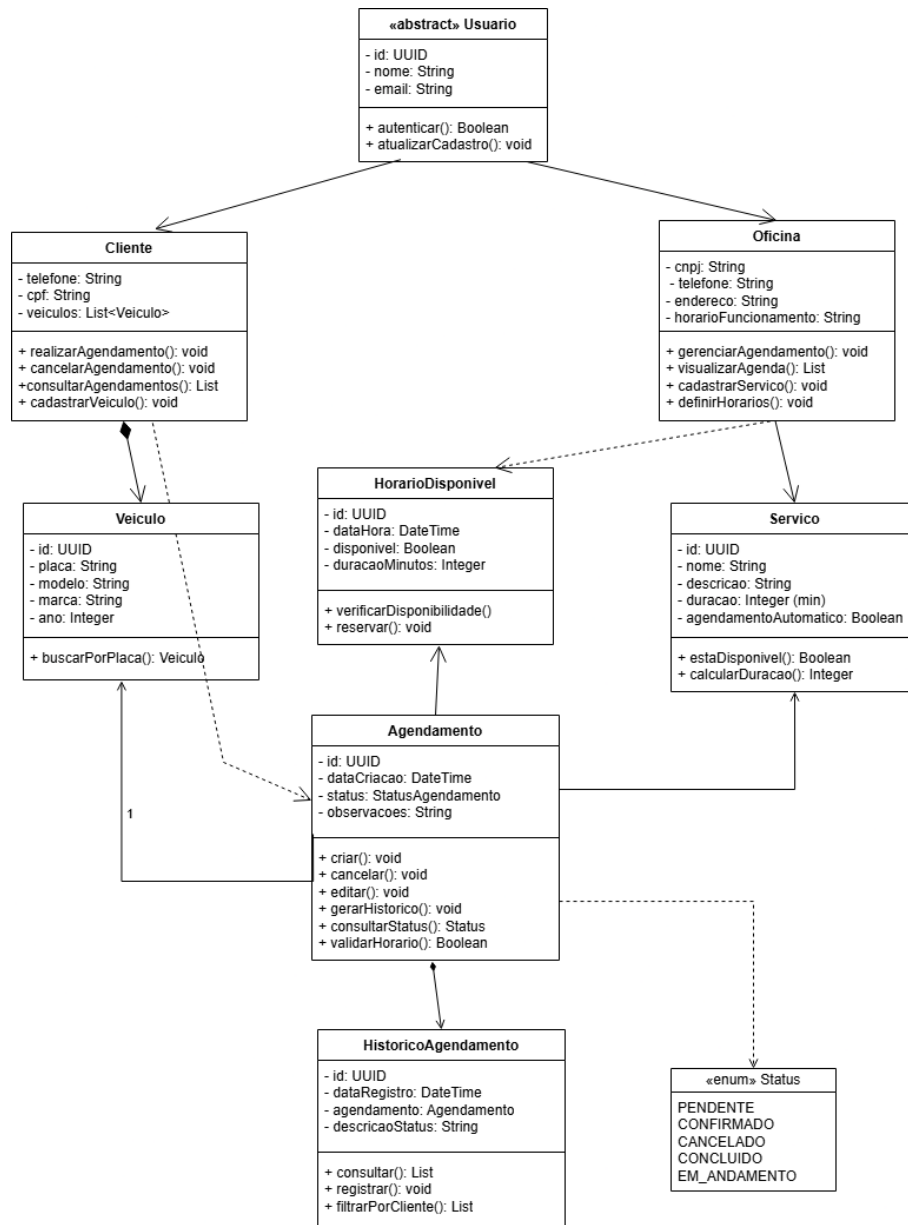


Figura 1. Diagrama de Classes do Sistema

5. Princípios SOLID

Os princípios SOLID [Sommerville 2019] orientam o desenvolvimento de software orientado a objetos com foco em manutenibilidade e extensibilidade. A seguir são descritos como cada princípio foi aplicado no sistema.

S — Single Responsibility Principle (Responsabilidade Única): Cada classe tem apenas uma razão para mudar. *Agendamento* cuida somente da lógica de agenda-

mento. *HorarioDisponivel* só gerencia disponibilidade de horários. *HistoricoAgendamento* só armazena registros históricos. Se a regra de negócio de cancelamento mudar, só *Agendamento* precisa ser alterado – nenhuma outra classe é impactada.

O — Open/Closed Principle (Aberto/Fechado): As classes estão abertas para extensão, mas fechadas para modificação. A classe abstrata *Usuario* exemplifica isso: ao criar *Cliente* e *Oficina* como subclasses, adicionamos comportamentos específicos sem mexer na classe base. Se surgir um novo perfil de usuário (por exemplo, Administrador), basta criar uma nova subclasse herdando de *Usuario*, sem alterar o código existente.

L — Liskov Substitution Principle (Substituição de Liskov): As subclasses podem substituir a classe base sem quebrar o sistema. Tanto *Cliente* quanto *Oficina* herdam de *Usuario* e implementam `autenticar()` e `atualizarCadastro()`. Em qualquer ponto do sistema que espera um *Usuario*, um *Cliente* ou uma *Oficina* pode ser usado no lugar, sem surpresas. Ambos se comportam como *Usuario* – apenas com responsabilidades adicionais.

I — Interface Segregation Principle (Segregação de Interfaces): Nenhuma classe é forçada a depender de métodos que não usa. *Cliente* tem `realizarAgendamento()` e `cancelarAgendamento()`, mas não tem `cadastrarServico()` ou `definirHorarios()` – isso é exclusivo de *Oficina*. Cada perfil só carrega os métodos que fazem sentido para ele, sem herdar um contrato inchado.

D — Dependency Inversion Principle (Inversão de Dependência): As classes de alto nível não dependem de detalhes concretos. *Agendamento* se associa a *Veiculo*, *HorarioDisponivel* e *Servico* por meio de referências abstratas (identificadores e tipos), não instanciando essas classes diretamente. Isso significa que a implementação de *Veiculo*, por exemplo, pode mudar internamente (busca por API externa, cache, etc.) sem que *Agendamento* precise ser reescrito.

6. Padrões de Projeto GoF

Para o desenvolvimento do sistema de agendamento de serviços automotivos foram selecionados três padrões de projeto GoF [Gamma et al. 2007], contemplando as categorias de criação, estrutural e comportamental. A escolha foi realizada com base nos requisitos funcionais, regras de negócio e arquitetura da aplicação, desenvolvida utilizando Python, Flask, React e banco de dados relacional.

6.1. Factory Method (Padrão de Criação)

6.1.1. Motivo da Escolha

O sistema possui diferentes tipos de usuários, sendo eles clientes e oficinas mecânicas. Embora compartilhem características em comum, cada tipo de usuário possui responsabilidades específicas dentro da aplicação. Dessa forma, tornou-se necessário um mecanismo que permitisse a criação desses objetos de maneira organizada e extensível [Gamma et al. 2007].

6.1.2. Problema Resolvido

Sem a utilização de um padrão de criação, a lógica de instanciação dos diferentes tipos de usuários ficaria distribuída pelo sistema através de estruturas condicionais, aumentando o acoplamento e dificultando futuras manutenções.

Com o Factory Method, a responsabilidade de criação dos objetos é centralizada em uma fábrica específica, responsável por determinar qual classe concreta deve ser instanciada de acordo com o tipo de usuário informado.

6.1.3. Impacto na Arquitetura

A utilização do Factory Method reduz o acoplamento entre as camadas da aplicação e as implementações concretas dos usuários. Além disso, facilita futuras expansões, permitindo a inclusão de novos perfis sem a necessidade de alterar regras já existentes.

O padrão também contribui para a aplicação do princípio Open/Closed Principle (OCP), tornando a arquitetura mais flexível e de fácil manutenção.

6.2. Facade (Padrão Estrutural)

6.2.1. Motivo da Escolha

O processo de agendamento envolve diversas entidades do sistema, como cliente, veículo, serviço, horário disponível, agendamento e histórico de atendimentos. Para evitar que as camadas superiores da aplicação precisem conhecer toda essa complexidade, optou-se pela utilização do padrão Facade [Gamma et al. 2007].

6.2.2. Problema Resolvido

Sem uma camada intermediária, os controladores da API precisariam interagir diretamente com diversas classes do domínio para realizar um único agendamento, aumentando a complexidade do código e dificultando sua manutenção.

O padrão Facade fornece uma interface simplificada para o processo de agendamento, centralizando validações e operações necessárias em um único ponto de acesso.

6.2.3. Impacto na Arquitetura

A adoção do Facade reduz a dependência entre os controladores Flask e as classes responsáveis pelas regras de negócio.

Dessa forma, as rotas da API possuem apenas a responsabilidade de receber e retornar dados, enquanto toda a lógica relacionada ao processo de agendamento permanece concentrada na camada de serviços da aplicação.

Essa abordagem melhora a organização do código, facilita a realização de testes e aumenta a manutenibilidade do sistema.

6.3. State (Padrão Comportamental)

6.3.1. Motivo da Escolha

Os agendamentos possuem um ciclo de vida composto por diferentes estados, tais como pendente, confirmado, em andamento, concluído e cancelado. Como cada estado possui comportamentos e transições específicas, o padrão State mostrou-se adequado para representar esse fluxo [Gamma et al. 2007].

6.3.2. Problema Resolvido

Uma implementação tradicional exigiria diversas estruturas condicionais para controlar as mudanças de estado e validar quais ações podem ser executadas em cada situação.

À medida que novos estados ou regras fossem adicionados ao sistema, a complexidade dessas estruturas aumentaria significativamente.

Com o padrão State, cada estado passa a ser representado por uma classe específica, responsável por definir os comportamentos permitidos naquele momento do ciclo de vida do agendamento.

6.3.3. Impacto na Arquitetura

A utilização do padrão State elimina a necessidade de grandes blocos condicionais relacionados ao status dos agendamentos.

Além disso, centraliza as regras de transição entre estados, tornando o código mais organizado, reutilizável e aderente às regras de negócio da aplicação.

Esse padrão também facilita futuras alterações, como a inclusão de novos estados ou fluxos de atendimento, minimizando impactos nas demais partes do sistema.

6.4. Conclusão

Os padrões Factory Method, Facade e State foram selecionados por apresentarem aderência direta aos requisitos e regras de negócio do sistema de agendamento para oficinas mecânicas [Gamma et al. 2007].

O Factory Method foi utilizado para centralizar a criação dos diferentes tipos de usuários. O Facade foi empregado para simplificar o processo de agendamento e reduzir o acoplamento entre as camadas da aplicação. Já o State foi adotado para representar adequadamente o ciclo de vida dos agendamentos e suas transições de estado.

A utilização desses padrões contribui para uma arquitetura mais organizada, flexível, escalável e alinhada às boas práticas de desenvolvimento orientado a objetos.

Referências

Booch, G., Rumbaugh, J., and Jacobson, I. (2005). *UML: Guia do Usuário*. Campus/Elsevier, Rio de Janeiro, 2 edition.

Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (2007). *Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos*. Bookman, Porto Alegre.

SEBRAE (2023). Setor automotivo: Desempenho e tendências. Disponível em: <https://www.sebrae.com.br>. Acesso em: 2025.

Sommerville, I. (2019). *Engenharia de Software*. Pearson Universidades, São Paulo, 10 edition.